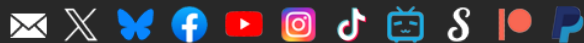
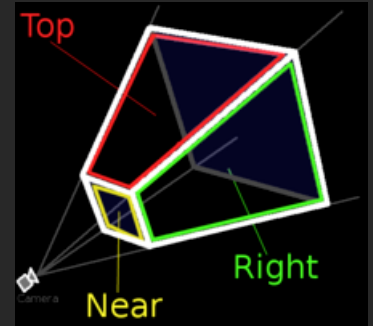


Inigo Quilez :: articles :: minimal frustum culling - 2010



You probably have done frustum culling in your 3d engine at some point. It's cheap to compute, and it can save the cost of rendering quite a big deal of objects. However, most size-coding demos and intros don't use it because it can involve quite a lot of code to do. As usual, things don't need to get as complex as most tutorials tell you. This article is about how to implement a fully functional simple, clean, with no dependency on external code, and small frustum culling. It can also serve as an example of how a 4 kilobyte demo coder might approach code optimization.



A frustum

So, to put things in context, the idea is to precompute some definition of the camera view frustum that will allow quick and inexpensive rejection of objects at rendering time. Let's assume you have a perspective based camera for this article (orthographic frustums are even easier to handle, so I leave that as an exercise to the reader - yay, I always wanted to say this). So, we have a camera defined by a field of view, an aspect ration, a near and far clipping distances. I assume that objects that will get tested against the frustum have bounding spheres (or even boxes) defined in camera space. So our camera is centered at the origin and is looking in the negative **z** direction (yeah, an OpenGL style coordinate system).

So, what we have to do is to construct the 6 planes that define the frustum. Now, you can do it the hard way, compute a projection matrix based on the OpenGL redbook, then copy from Nehe's tutorials the code to extract the planes from the matrix, and then normalize them. But we will do it another way - we are gonna construct the planes directly, because we perfectly know our frustum definition: the pass thru the origin, they point perpendicular to the field of view (fov) lines, and are parallel to the xyz axes. Besides, we don't need to normalize them, because we are gonna use the frustum only as an in/out/intersect detector for our geometry, so we don't need to know how far an object was from being inside or not, only if it is. So we can for sure avoid all the plane normalization code. Basically, we get this:

```
void iqFrustumF_CreatePerspective( float *frus, float fovy, float aspect, float z1
{
```

```

const float an = fovy * (3.141592653589f/180.0f);
const float si = sinf(an);
const float co = cosf(an);
frus[ 0] =  0.0f; frus[ 1] = -co;      frus[ 2] =  si;      frus[ 3] =  0.0f;
frus[ 4] =  0.0f; frus[ 5] =  co;      frus[ 6] =  si;      frus[ 7] =  0.0f;
frus[ 8] =  co;   frus[ 9] =  0.0f;    frus[10] = si*aspect; frus[11] =  0.0f;
frus[12] = -co;   frus[13] =  0.0f;    frus[14] = si*aspect; frus[15] =  0.0f;
frus[16] =  0.0f; frus[17] =  0.0f;    frus[18] =  1.0f;    frus[19] = zfar;
frus[20] =  0.0f; frus[21] =  0.0f;    frus[22] = -1.0f;    frus[23] = -znear;
}

```

The first line is the first clipping plane, and it's made of 4 floats, as all 3D planes. It passes thru the origin (third coefficient is 0.0), it's perpendicular to the **yz** plane, and it points IN the frustum volume (so, it's the top plane). Note that its normal vector is (0, -co, si), which accidentally has length 1 (oops!), and it makes an angle of $\text{atan}(si/co) = \text{atan}(\sin(an)/\cos(an)) = \text{atan}(\tan(an)) = an$ radians with the z axis, exactly as we want. Similar reasoning goes for the following 3 planes. The last two planes, the near and far clipping planes, are trivial: far clipping plane is defined as $z=-zfar$ (remember, **z** is negative when looking forward) so the equation is $z+zfar=0$, ie, (1,0,0,zfar). Same for the near clipping plane (both have unit length normals already, oops!).

The frustum is stored in 24 floats, which can be a local array in the callee function. No need for structs or other stuff.

Another way

For the record, one should be able to arrive to this same code/formulas as a simplification process from a more general frustum culling source code. Let's see how. Let's start by a Nehe-style frustum culling code:

```

void iqFrustumF_CreateFrustum( float *frustum, float fovy, float aspect, float znear, float zfar )
{
    // extract corner points
    const float top    = znear * tanf(fovy * 3.141592653589f/180.0f );
    const float bottom = -top;
    const float left   = bottom * aspect;
    const float right  = top * aspect;

    // construct an OpenGL-style projection matrix
    float x = (2.0f * znear) / (right - left);
    float y = (2.0f * znear) / (top - bottom);
    float a = (right + left) / (right - left);
    float b = (top + bottom) / (top - bottom);
}

```

```

float c = -(zfar + znear) / ( zfar - znear);
float d = -(2.0f * zfar * znear) / (zfar - znear);

float matrix[16];
matrix[ 0] = x;      matrix[ 1] = 0.0f;  matrix[ 2] = a;      matrix[ 3] = 0.0f;
matrix[ 4] = 0.0f;  matrix[ 5] = y;      matrix[ 6] = b;      matrix[ 7] = 0.0f;
matrix[ 8] = 0.0f;  matrix[ 9] = 0.0f;  matrix[10] = c;      matrix[11] = d;
matrix[12] = 0.0f;  matrix[13] = 0.0f;  matrix[14] = -1.0f;  matrix[15] = 0.0f;

// extract planes from the projection matrix
frustum[ 0] = matrix[12] - matrix[ 0];
frustum[ 1] = matrix[13] - matrix[ 1];
frustum[ 2] = matrix[14] - matrix[ 2];
frustum[ 3] = matrix[15] - matrix[ 3];

frustum[ 4] = matrix[12] + matrix[ 0];
frustum[ 5] = matrix[13] + matrix[ 1];
frustum[ 6] = matrix[14] + matrix[ 2];
frustum[ 7] = matrix[15] + matrix[ 3];

frustum[ 8] = matrix[12] + matrix[ 4];
frustum[ 9] = matrix[13] + matrix[ 5];
frustum[10] = matrix[14] + matrix[ 6];
frustum[11] = matrix[15] + matrix[ 7];

frustum[12] = matrix[12] - matrix[ 4];
frustum[13] = matrix[13] - matrix[ 5];
frustum[14] = matrix[14] - matrix[ 6];
frustum[15] = matrix[15] - matrix[ 7];

frustum[16] = matrix[12] - matrix[ 8];
frustum[17] = matrix[13] - matrix[ 9];
frustum[18] = matrix[14] - matrix[10];
frustum[19] = matrix[15] - matrix[11];

frustum[20] = matrix[12] + matrix[ 8];
frustum[21] = matrix[13] + matrix[ 9];
frustum[22] = matrix[14] + matrix[10];
frustum[23] = matrix[15] + matrix[11];

// normalize planes
for( int i=0; i<6; i++, ptr+=4 )
{
    const float im = 1.0f/sqrtf( frustum[4*i+0]*frustum[4*i+0] +
                                frustum[4*i+1]*frustum[4*i+1] +
                                frustum[4*i+2]*frustum[4*i+2] );

    frustum[4*i+0] *= im;
    frustum[4*i+1] *= im;
    frustum[4*i+2] *= im;
    frustum[4*i+3] *= im;
}

```

```

    }
}

```

As said, we don't need to normalize the planes for our purpose, so we can get rid of the last block of code. Next, we directly replace the left/right/top/bottom quantities in the definition of the matrix. Since it's a symmetric frustum and therefore left=-right and bottom=-top, and since znear can be factored out, we get some nice cancelations in the terms:

```

void iqFrustumF_CreateFrustum( float *frustum, float fovy, float aspect, float znear, float zfar )
{
    const float taa = tanf(fovy * 3.141592653589f/180.0f );

    // construct an OpenGL-style projection matrix
    float y = 1.0f / taa;
    float x = aspect * 1.0f / taa;
    float a = 0.0f;
    float b = 0.0f;
    float c = -(zfar + znear) / ( zfar - znear);
    float d = -(2.0f * zfar * znear) / (zfar - znear);

    float matrix[16];
    matrix[ 0] = x;      matrix[ 1] = 0.0f;  matrix[ 2] = a;      matrix[ 3] = 0.0f;
    matrix[ 4] = 0.0f;  matrix[ 5] = y;      matrix[ 6] = b;      matrix[ 7] = 0.0f;
    matrix[ 8] = 0.0f;  matrix[ 9] = 0.0f;  matrix[10] = c;      matrix[11] = d;
    matrix[12] = 0.0f;  matrix[13] = 0.0f;  matrix[14] = -1.0f; matrix[15] = 0.0f;

    // extract planes from the projection matrix
    frustum[ 0] = matrix[12] - matrix[ 0];
    frustum[ 1] = matrix[13] - matrix[ 1];
    frustum[ 2] = matrix[14] - matrix[ 2];
    frustum[ 3] = matrix[15] - matrix[ 3];

    frustum[ 4] = matrix[12] + matrix[ 0];
    frustum[ 5] = matrix[13] + matrix[ 1];
    frustum[ 6] = matrix[14] + matrix[ 2];
    frustum[ 7] = matrix[15] + matrix[ 3];

    frustum[ 8] = matrix[12] + matrix[ 4];
    frustum[ 9] = matrix[13] + matrix[ 5];
    frustum[10] = matrix[14] + matrix[ 6];
    frustum[11] = matrix[15] + matrix[ 7];

    frustum[12] = matrix[12] - matrix[ 4];
    frustum[13] = matrix[13] - matrix[ 5];
    frustum[14] = matrix[14] - matrix[ 6];
    frustum[15] = matrix[15] - matrix[ 7];

    frustum[16] = matrix[12] - matrix[ 8];
}

```

```

    frustum[17] = matrix[13] - matrix[ 9];
    frustum[18] = matrix[14] - matrix[10];
    frustum[19] = matrix[15] - matrix[11];

    frustum[20] = matrix[12] + matrix[ 8];
    frustum[21] = matrix[13] + matrix[ 9];
    frustum[22] = matrix[14] + matrix[10];
    frustum[23] = matrix[15] + matrix[11];
}

```

Next, we introduce the terms directly in the plane extraction code, and get:

```

void iqFrustumF_CreateFrustum( float *frustum, float fovy, float aspect, float znear, float zfar )
{
    const float taa = tanf(fovy * 3.141592653589f/180.0f );

    // construct an OpenGL-style projection matrix
    float y = 1.0f / taa;
    float x = aspect * 1.0f / taa;
    float c = -(zfar + znear) / ( zfar - znear);
    float d = -(2.0f * zfar * znear) / (zfar - znear);

    // extract planes from the projection matrix
    frustum[ 0] = x;      frustum[ 1] = 0.0f;   frustum[ 2] = 1.0f;      frustum[ 3] = 0.0f;
    frustum[ 4] = x;      frustum[ 5] = 0.0f;   frustum[ 6] = -1.0f;     frustum[ 7] = 0.0f;
    frustum[ 8] = 0.0f;   frustum[ 9] = y;      frustum[10] = -1.0f;     frustum[11] = 0.0f;
    frustum[12] = 0.0f;   frustum[13] = -y;     frustum[14] = -1.0f;     frustum[15] = 0.0f;
    frustum[16] = 0.0f;   frustum[17] = 0.0f;   frustum[18] = -1.0f - c; frustum[19] = d;
    frustum[20] = 0.0f;   frustum[21] = 0.0f;   frustum[22] = -1.0f + c; frustum[23] = d;
}

```

Much better already. Now we apply some algebra to c , d , $-1-c$, $1+c$ and all these terms in the 4th and 5th planes. We see that $-1-c = 2n/(f-n)$ and that $-1+c = -2f/(f-n)$. Since $d = -2fn/(f-n)$, we can multiply the whole plane equations by $(f-n)/2$ and get:

```

void iqFrustumF_CreateFrustum( float *frustum, float fovy, float aspect, float znear, float zfar )
{
    const float taa = tanf(fovy * 3.141592653589f/180.0f );

    float y = 1.0f / taa;
    float x = aspect * 1.0f / taa;

    frustum[ 0] = x;      frustum[ 1] = 0.0f;   frustum[ 2] = 1.0f;      frustum[ 3] = 0.0f;
    frustum[ 4] = x;      frustum[ 5] = 0.0f;   frustum[ 6] = -1.0f;     frustum[ 7] = 0.0f;
    frustum[ 8] = 0.0f;   frustum[ 9] = y;      frustum[10] = -1.0f;     frustum[11] = 0.0f;
    frustum[12] = 0.0f;   frustum[13] = -y;     frustum[14] = -1.0f;     frustum[15] = 0.0f;
    frustum[16] = 0.0f;   frustum[17] = 0.0f;   frustum[18] = znear;     frustum[19] = d;
}

```

```

    frustum[20] = 0.0f;   frustum[21] = 0.0f;   frustum[22] = -zfar;   frustum[23] = 0.0f;
}

```

Next is to realize that $\tan() = \sin()/\cos()$, therefore $x = \cos/\sin$, and similar stuff goes for y , so we can write:

```

void iqFrustumF_CreateFrustum( float *frustum, float fovy, float aspect, float znear, float zfar )
{
    const float an = fovy * (3.141592653589f/180.0f);
    const float si = sinf(an);
    const float co = cosf(an);

    frustum[ 0] = aspect*co/si; frustum[ 1] = 0.0f;   frustum[ 2] = 1.0f;   frustum[ 3] = 0.0f;
    frustum[ 4] = aspect*co/si; frustum[ 5] = 0.0f;   frustum[ 6] = -1.0f;  frustum[ 7] = 0.0f;
    frustum[ 8] = 0.0f;        frustum[ 9] = co/si;   frustum[10] = -1.0f;  frustum[11] = 0.0f;
    frustum[12] = 0.0f;        frustum[13] = -co/si;  frustum[14] = -1.0f;  frustum[15] = 0.0f;
    frustum[16] = 0.0f;        frustum[17] = 0.0f;   frustum[18] = znear;   frustum[19] = zfar;
    frustum[20] = 0.0f;        frustum[21] = 0.0f;   frustum[22] = -zfar;   frustum[23] = 0.0f;
}

```

Last step is to divide the 4th plane by $znear$, and the 5th by $zfar$, then to multiply the rest by si , and we pretty much arrive to the final code, which is much smaller than the original one, and much more intuitive, as you can see the plane equations directly in the code.

```

void iqFrustumF_CreatePerspective( float *frus, float fovy, float aspect, float znear, float zfar )
{
    const float an = fovy * (3.141592653589f/180.0f);
    const float si = sinf(an);
    const float co = cosf(an);

    frus[ 0] = 0.0f; frus[ 1] = -co;   frus[ 2] = si;   frus[ 3] = 0.0f;
    frus[ 4] = 0.0f; frus[ 5] = co;    frus[ 6] = si;   frus[ 7] = 0.0f;
    frus[ 8] = co;   frus[ 9] = 0.0f;  frus[10] = si*aspect; frus[11] = 0.0f;
    frus[12] = -co;  frus[13] = 0.0f;  frus[14] = si*aspect; frus[15] = 0.0f;
    frus[16] = 0.0f; frus[17] = 0.0f;  frus[18] = 1.0f;   frus[19] = zfar;
    frus[20] = 0.0f; frus[21] = 0.0f;  frus[22] = -1.0f;  frus[23] = -znear;
}

```

How to use it

As said, a minimal frustum is just this, 24 floats (6 planes). Therefore, you could use it like this:

```

float frustum[24];
iqFrustumF_CreatePerspective( frustum, 50.0f, 1.333f, 0.1f, 1000.0f );

for( int i=0; i < numObjects; i++ )

```

```
{  
    if( iqFrustumF_BoxInside( frustum, box+6*i )!=0 )  
    {  
        addObjectToRenderList( i );  
    }  
}
```

the *iqFrustumF_BoxInside()* function intersects a camera aligned bounding box with the frustum. In a real example you would probably want to actually transform the frustum to object space instead (and probably do it through the original frustum construction code - ehem.....) or use bounding spheres instead which are easily transformed to camera space. In a 4k intro (or even 64k demo), you would probably go for the second :)

inigo quilez - learning computer graphics since 1994